Stage 1 Step-by-Step Practical & Conceptual Recap

1. Express Routing Rules (app.get)

- **What We Built:**
  app.get('/', (req, res) => { ... });
  app.get('/health', (req, res) => { ... });

- **Sysadmin / Network Analogy:** Express routing functions like an **Nginx Reverse Proxy or Firewall Access Control List (ACL)**. When an inbound TCP packet hits port 3000, the routing engine evaluates the request header against two criteria:
  1. **HTTP Method / Verb:** GET (Retrieval)
  2. **URI Path:** / or /health

If a match is found, traffic is routed to the designated request-handler function. If no match exists, Express automatically drops through to its default rule and returns an HTTP 404 Not Found response.

2. Structured JSON Payload Serialization (res.json vs res.send)

- **What We Built:** Replacing plain text output with structured JSON responses:
  res.json({ name: "Task API", version: "1.0", endpoints: ["/tasks"] });

- **Sysadmin / Network Analogy:** In Stage 0, res.send() transmitted unstructured text (Content-Type: text/html). In Stage 1, res.json() automatically sets the MIME header to Content-Type: application/json; charset=utf-8.
- **Why it matters:** In automated B2B integrations (like invoice extraction pipelines for Kenyan SMEs), client systems cannot easily parse arbitrary text strings. Delivering predictable JSON structures allows downstream automation scripts and database adapters to safely parse payloads into typed data fields.

3. Service Discovery & Banners (GET /)

- **What We Built:** A root endpoint returning API metadata and available endpoints.
- **Sysadmin / Network Analogy:** This acts like an **SNMP Discovery Query** or an **SSH/FTP System Identification Banner**. When an external client or orchestration service hits your root URL, it receives an automated system manifesto detailing what the server is (Task API) and where its operational resources reside (/tasks).

4. Operational Health Checks & Liveness Probes (GET

/health)

- **What We Built:** A lightweight endpoint returning {"status": "ok"} with HTTP 200.
- **Sysadmin / Network Analogy:** This functions as an enterprise **System Heartbeat Probe or ICMP Keep-Alive**. Production load balancers (e.g., Nginx, HAProxy, AWS ALB) and monitoring daemons (Nagios, Zabbix, Docker) periodically ping /health.
- **Operational Impact:** If your application process deadlocks or crashes, the monitor detects failed /health checks (e.g., timeouts or 500 errors) and automatically reroutes traffic or restarts the daemon process.

5. Memory Buffering vs Disk State (Ctrl + S & Daemon Restarts)

- **What Happened:** When you modified server.js, curl initially kept returning old Stage 0 data until you saved the file and restarted Node.
- **Sysadmin / Network Analogy:**
  - Unsaved edits in VS Code live in volatile **RAM buffers** (marked by the white dot/M status).
  - Node loads the binary script into execution memory only upon startup. Running changes require flushing the active daemon (Ctrl + C) and re-executing node server.js to force the runtime to read the updated bytecode from disk.

6. Repository Hygiene & Version Control Filtering (.gitignore)

- **What We Built:** Creating .gitignore containing node_modules/ and running git rm -r --cached node_modules.
- **Sysadmin / Network Analogy:** Committing node_modules/ into Git is equivalent to backing up temp files (/tmp/) or compiled binary libraries inside a system configuration snapshot repository. Excluding dependencies keeps your codebase lightweight, secure, and compliant with clean repository management standards.